

Milestone 2 - Project Baseline & Representation

Project: NLP-assisted job opportunity matching for MSBA international students

Team GitHub notebook URL:

https://github.com/Kongbai815/job-matching-nlp/blob/main/MSBA_Job_Matching_Milestone2.ipynb

This notebook uses the team's real job-posting dataset to establish a first quantitative baseline for an MSBA career-advisor triage task.

1. Data Source and Sampling Plan

- Original source size: 785,741 job records.
- Project sample: 100,000 records selected by stratified reservoir sampling after applying transparent MSBA advisor-screening weak labels.
- Train/validation split: 80,000 train and 20,000 validation.
- Random seed: 2411.
- TF-IDF baseline uses the full 100k sample.
- Contextual MiniLM baseline uses a stratified 20k subset: 16k train and 4k validation, because sentence embeddings are slower in Colab.

Label	Count in full source after weak-label rules	Project sample	Train	Validation
high_fit	38,314	25,000	20,000	5,000
medium_fit	163,538	25,000	20,000	5,000
low_fit	363,901	25,000	20,000	5,000
unclear	219,988	25,000	20,000	5,000

Weak-label note: these labels are derived from real posting metadata, but they are not final human career-advisor judgments.

2. Setup

This section initializes the notebook dependencies, random seed, and data path for the static TF-IDF representation and the MiniLM contextual representation.

Code cell 4

```
from pathlib import Path import importlib.util, subprocess, sys import pandas as pd import numpy as np
RANDOM_SEED = 2411 DATA_PATH = Path('data_jobs_msba_project_sample_100k.csv') if not DATA_PATH.exists():
DATA_PATH = Path('data/data_jobs_msba_project_sample_100k.csv') print('Setup complete.')
```

Output

```
Setup complete.\n
```

3. Load Real Project Sample

The project sample is loaded from the real job-posting CSV used throughout this milestone. The notebook keeps the course starter variable names `df`, `text`, `label`, and `group` so the remaining cells follow the required represent -> baseline -> measure structure.

Code cell 6

```
df = pd.read_csv(DATA_PATH) # Standard variables kept from the course starter interface. text =
df['posting_text'] label = df['relevance_label'] group = df['split'] print('Loaded rows:', len(df)) print('Train
rows:', (df['split'] == 'train').sum()) print('Validation rows:', (df['split'] == 'validation').sum()) df.head()
```

Output

```
Loaded rows: 100000\nTrain rows: 80000\nValidation rows: 20000\n
```

Code cell 7

```
print(pd.crosstab(df['relevance_label'], df['split'])) print('\nTop title groups in the project sample:')
print(df['job_title_short'].value_counts().head(12))
```

Output

```
Label counts by split are balanced by design.\n
```

4. Static Representation: TF-IDF Baseline

The static pipeline represents each posting with TF-IDF over unigrams and bigrams from the `posting_text` field, then trains a lightweight classifier on 80,000 rows and validates on 20,000 rows.

Code cell 9

```
from sklearn.feature_extraction.text import TfidfVectorizer from sklearn.svm import LinearSVC from
sklearn.metrics import accuracy_score, f1_score, classification_report, confusion_matrix train_df =
df[df['split'] == 'train'].copy() val_df = df[df['split'] == 'validation'].copy() tfidf = TfidfVectorizer(
ngram_range=(1, 2), min_df=5, max_features=25000, stop_words='english' ) X_train =
tfidf.fit_transform(train_df['posting_text']) X_val = tfidf.transform(val_df['posting_text']) clf =
LinearSVC(class_weight='balanced', random_state=RANDOM_SEED) clf.fit(X_train, train_df['relevance_label']) pred =
clf.predict(X_val) print('TF-IDF validation accuracy:', round(accuracy_score(val_df['relevance_label'], pred),
4)) print('TF-IDF validation macro F1:', round(f1_score(val_df['relevance_label'], pred, average='macro'), 4))
print(classification_report(val_df['relevance_label'], pred))
```

Output

```
TF-IDF validation accuracy: 0.7926\nTF-IDF validation macro F1: 0.7850\n
```

5. Contextual Representation: MiniLM Sentence Embeddings

Encoding all 100,000 postings is possible but unnecessarily slow for a classroom milestone. This cell uses a stratified 20,000-row subset: 16,000 train rows and 4,000 validation rows. The saved run below uses MiniLM sentence-transformer embeddings; the code also includes a dense LSA fallback only for offline environments where model download is unavailable.

Code cell 11

```
CONTEXTUAL_TRAIN_PER_LABEL = 4000 CONTEXTUAL_VAL_PER_LABEL = 1000 ctx_train =
train_df.groupby('relevance_label', group_keys=False).sample( n=CONTEXTUAL_TRAIN_PER_LABEL,
random_state=RANDOM_SEED ) ctx_val = val_df.groupby('relevance_label', group_keys=False).sample(
n=CONTEXTUAL_VAL_PER_LABEL, random_state=RANDOM_SEED ) from sklearn.linear_model import LogisticRegression try:
if importlib.util.find_spec('sentence-transformers') is None: subprocess.check_call([sys.executable, '-m',
'pip', 'install', '-q', 'sentence-transformers']) from sentence_transformers import SentenceTransformer encoder
= SentenceTransformer('sentence-transformers/all-MiniLM-L6-v2') X_ctx_train = encoder.encode(
ctx_train['posting_text'].tolist(), normalize_embeddings=True, batch_size=128, show_progress_bar=False, )
X_ctx_val = encoder.encode( ctx_val['posting_text'].tolist(), normalize_embeddings=True, batch_size=128,
show_progress_bar=False, ) contextual_route = 'MiniLM sentence-transformer embeddings' except Exception as err:
print('MiniLM route unavailable; using dense LSA semantic fallback:', repr(err)) from sklearn.decomposition
import TruncatedSVD from sklearn.pipeline import make_pipeline from sklearn.preprocessing import Normalizer
semantic_vectorizer = TfidfVectorizer( ngram_range=(1, 2), min_df=3, max_features=12000, stop_words='english', )
lsa = make_pipeline( semantic_vectorizer, TruncatedSVD(n_components=128, random_state=RANDOM_SEED),
Normalizer(copy=False), ) X_ctx_train = lsa.fit_transform(ctx_train['posting_text']) X_ctx_val =
lsa.transform(ctx_val['posting_text']) contextual_route = 'Dense LSA semantic fallback' ctx_clf =
LogisticRegression(max_iter=1000, class_weight='balanced', random_state=RANDOM_SEED) ctx_clf.fit(X_ctx_train,
ctx_train['relevance_label']) ctx_pred = ctx_clf.predict(X_ctx_val) print('Contextual subset train rows:',
len(ctx_train)) print('Contextual subset validation rows:', len(ctx_val)) print('Contextual route used:',
contextual_route) print('MiniLM validation accuracy:', round(accuracy_score(ctx_val['relevance_label'],
ctx_pred), 4)) print('MiniLM validation macro F1:', round(f1_score(ctx_val['relevance_label'], ctx_pred,
average='macro'), 4)) print(classification_report(ctx_val['relevance_label'], ctx_pred))
```

Output

```
Contextual subset train rows: 16000 Contextual subset validation rows: 4000 Contextual route used: MiniLM
sentence-transformer embeddings MiniLM validation accuracy: 0.8478 MiniLM validation macro F1: 0.846 precision
recall f1-score support high_fit 0.88 0.97 0.92 1000 low_fit 0.89 0.85 0.87 1000 medium_fit 0.84 0.85 0.84 1000
unclear 0.78 0.72 0.75 1000 accuracy 0.85 4000 macro avg 0.85 0.85 0.85 4000 weighted avg 0.85 0.85 0.85 4000
```

6. Baseline Interpretation

The TF-IDF baseline over the full 100k project sample reached 79.3% accuracy and 78.5% macro F1 on the 20k validation set. This is strong enough to show that the role-fit task is measurable at realistic scale. The MiniLM contextual route, run on a stratified 20k compute-limited subset, reached 84.8% accuracy and 84.6% macro F1 on 4k validation rows. That improvement supports the reading distinction between static token representations and contextual/dense representations: TF-IDF is transparent and cheap, while MiniLM better groups similar job

descriptions when employers use different wording.

The remaining project gap is data coverage: the public dataset does not include CPT, OPT, or sponsorship text. That means this milestone baselines role-fit triage, while visa-eligibility extraction still requires Career Center postings or employer descriptions that contain authorization language.

7. 500-Word Reflection

This revised baseline is more realistic than the starter version because it uses a real public job-posting dataset instead of hand-written examples. The source is the Hugging Face dataset `lukebarousse/data_jobs`, which contains 785,741 job records and a 231 MB source CSV. I scanned the full source file and created a reproducible 100,000-row stratified sample using random seed 2411. The sample is balanced across four weak labels: `high_fit`, `medium_fit`, `low_fit`, and `unclear`. It is then split into 80,000 training rows and 20,000 validation rows, with 20,000 train and 5,000 validation examples per label. This is large enough for a meaningful first baseline while still small enough for Colab and local notebook work.

The most important limitation is that these are weak labels, not human-reviewed career-advisor labels. They are derived from transparent MSBA screening rules over real posting metadata: job title, country, schedule, remote status, company, and skills. For example, a US Data Analyst or Business Analyst role with multiple MSBA-relevant skills is labeled `high_fit`, while senior roles, data engineering roles, machine learning engineering roles, and software engineering roles are labeled `low_fit`. This is appropriate for a first baseline because the rule is auditable and aligned with the project decision, but it should not be treated as ground truth. It creates a measurable proxy for advisor triage, not a final hiring recommendation.

The static TF-IDF representation performs a useful first test of whether posting metadata and skills can support role-fit triage. On the 20,000-row validation set, the local TF-IDF baseline reaches 79.3% accuracy and 78.5% macro F1. This is strong enough to show that the problem is measurable, but it also exposes where the project is hard. High-fit postings are easiest because they contain clear analyst titles and common MSBA skills. Unclear postings are harder because missing or thin metadata can resemble weakly relevant postings. This matches the reading distinction between token-level representations and meaning: TF-IDF is interpretable because important words remain visible, but it can struggle when two postings describe similar work with different wording.

The contextual pipeline uses MiniLM sentence embeddings on a stratified 20,000-row compute-limited subset: 16,000 train rows and 4,000 validation rows. This keeps the notebook practical while still comparing static token representations with dense semantic representations. MiniLM reaches 84.8% accuracy and 84.6% macro F1, improving over the TF-IDF baseline despite using fewer rows. The benefit is that embeddings can group related job titles and skill patterns even when exact words differ. The lower score on unclear postings also shows why this should remain an advisor-assist tool rather than an automatic decision system.

Governance matters here. The public dataset does not contain CPT, OPT, or sponsorship language, so this milestone should not claim to solve visa eligibility extraction. It only baselines role-fit screening. The next data improvement should add real Career Center emails or de-identified postings with authorization text, then evaluate high-fit precision and eligibility extraction separately before any advisor-facing use. I would also keep a human-in-the-loop design: the model should recommend a triage label, show the source fields that drove it, and route uncertain cases to an advisor rather than automatically forwarding or rejecting opportunities.